🎤 Recommended Self-Introduction — 60–90 Seconds

"Good morning, and thank you for giving me this opportunity to introduce myself.

My name is **Satheesh Kumar**, and I am a **Senior Microsoft .NET Full Stack and Azure Developer with over 12 years of experience** in software development.

My core expertise is in **C#, .NET, ASP.NET, Web API, Angular, SQL Server, and Microsoft Azure**. I have extensive experience in developing enterprise web applications, building RESTful APIs, and working on scalable backend and frontend solutions.

Most recently, I worked with **Quadrant Technologies as a Senior Technical Analyst from December 2021 to July 2026**, where I worked on a **Microsoft project related to Azure Cosmos DB**. My experience there included **Azure Cosmos DB, Azure Functions, Azure Storage, Managed Identity, Azure RBAC, Azure DevOps, Bicep, and ARM Templates**.

Before that, I worked as a **Senior .NET Developer**, where I developed enterprise **ASP.NET/MVC applications and REST APIs** for international clients.

Throughout my career, I have been involved in **application development, API development, production support, troubleshooting, and performance optimization**. I particularly enjoy solving complex technical problems and designing scalable solutions using .NET and Azure.

Now, I am looking for a challenging Senior .NET, Azure, or Engineering Lead opportunity where I can contribute my technical expertise and take on greater responsibilities.

That's a brief introduction about me. Thank you."

⭐ Short Version — Best for HR Round

For an HR interview, I recommend keeping it around **45–60 seconds**:

"Hi, I'm Satheesh Kumar. I'm a Senior Microsoft .NET Full Stack and Azure Developer with over 12 years of experience.

My primary expertise is in **C#, .NET, ASP.NET, Web API, Angular, SQL Server, and Microsoft Azure**.

Most recently, I worked with **Quadrant Technologies as a Senior Technical Analyst from December 2021 to July 2026**, where I worked on a **Microsoft Azure Cosmos DB project** and gained strong experience with Azure Cosmos DB, Azure Functions, Azure Storage, Managed Identity, RBAC, Azure DevOps, Bicep, and ARM Templates.

Prior to that, I worked as a Senior .NET Developer, developing enterprise ASP.NET/MVC applications and REST APIs.

I have strong experience in enterprise application development, API development, production support, troubleshooting, and performance optimization.

I'm now looking for a challenging Senior .NET and Azure role where I can contribute my technical experience and grow into greater technical and leadership responsibilities."

👨‍💼 For the Engineering Lead Interview

Since you previously mentioned an **Engineering Lead – .NET + Azure** opportunity, I would use a slightly different version:

"Good morning. I'm Satheesh Kumar, a Senior .NET Full Stack and Azure Developer with over 12 years of experience.

My core technical expertise includes **C#, .NET, Web API, ASP.NET, Angular, SQL Server, and Microsoft Azure**.

My most recent experience was with **Quadrant Technologies, where I worked from December 2021 to July 2026 as a Senior Technical Analyst on a Microsoft Azure Cosmos DB project**. I worked extensively with **Azure Cosmos DB, Azure Functions, Azure Storage, Managed Identity, Azure RBAC, Azure DevOps, Bicep, and ARM Templates**.

Prior to that, I worked as a Senior .NET Developer, developing enterprise ASP.NET/MVC applications and REST APIs for international clients.

Over the years, I have worked across the complete application development lifecycle, including **requirement understanding, design, development, API integration, troubleshooting, production support, and performance optimization**.

I enjoy working on complex technical problems, collaborating with teams, and helping deliver scalable and reliable solutions. **At this stage of my career, I'm looking for an Engineering Lead or Senior Technical role where I can leverage my .NET and Azure expertise while taking on greater technical ownership and team responsibilities.**

Thank you. I'm happy to explain my recent project and technical experience in detail."

Important

Because you are **currently between roles**, avoid saying:

❌ "I am currently working at Quadrant Technologies."

Instead say:

✅ **"Most recently, I worked with Quadrant Technologies from December 2021 to July 2026."**

This sounds professional and immediately makes your employment status clear without unnecessarily emphasizing that you are currently not employed.

in contunuation of this interview process, if asked a question that I don't remember or don't know what should be my response?

Yes. This is **very common in senior-level interviews**, and the way you respond can matter almost as much as knowing the answer.

For your **Senior .NET / Azure / Engineering Lead** interviews, don't panic or try to bluff. The goal is to demonstrate **honesty, confidence, problem-solving ability, and willingness to learn**.

1. If you know it, but you can't remember at that moment

Don't say:

❌ "I don't know."

Instead:

"I'm familiar with this concept, but I can't recall the exact details at the moment. Let me explain what I remember about it..."

Then explain whatever you genuinely know.

Example

Interviewer:
"What is the difference between Cosmos DB consistency levels?"

You:

"I'm familiar with Cosmos DB consistency models, but I can't recall all the exact details at the moment. I do remember that they provide different trade-offs between consistency, latency, and availability. I would need to verify the exact behavior of each level before implementing it in production."

This is much better than guessing.

---

2. If you genuinely don't know

Be honest, but **don't stop at "I don't know."**

Use:

"I haven't worked directly with that technology/concept, so I don't want to give you an incorrect answer. However, based on my experience with similar technologies, my understanding would be..."

If you can connect it to something you know, do so.

Example

Interviewer:
"Have you worked with Kubernetes?"

You:

"I haven't worked directly with Kubernetes in a production environment, so I don't want to claim hands-on experience. My primary cloud experience has been with Azure services such as Cosmos DB, Azure Functions, Azure Storage and Azure DevOps. I understand the purpose of Kubernetes for container orchestration, but I would need some hands-on experience to discuss it in depth."

That's a **strong professional answer**.

---

3. If you understand the concept but don't remember the terminology

This happens frequently.

Say:

"I know the concept you're referring to, but I'm not recalling the exact terminology. Let me explain it from my understanding."

Then explain it.

This is especially useful for **design patterns, Azure services, C# features, and architectural concepts**.

---

4. If the question is unclear

Don't immediately answer.

Ask the interviewer to clarify.

"Could you please clarify the scenario you're referring to?"

or:

"Are you asking this from a .NET implementation perspective or from an architecture perspective?"

This is actually a **senior-level behavior**.

For example:

Interviewer:
"How would you handle caching?"

You can ask:

"Would you like me to answer that for a single application instance, or for a distributed application running across multiple instances?"

Now you have demonstrated architectural thinking.

---

5. If you need a moment to think

It's perfectly acceptable to pause.

Use:

"That's a good question. Let me think about the best way to approach it."

Take a few seconds.

Then structure your answer.

Don't fill the silence with:

"Actually... basically... you know... I think..."

A short pause makes you look **thoughtful rather than nervous**.

---

6. If you give an answer and then realize it may be wrong

Correct yourself.

"Let me correct that. I believe I misspoke in my previous answer. The correct approach is..."

This is much better than continuing with an incorrect answer.

Senior developers are expected to be able to **recognize and correct mistakes**.

---

7. If they ask something completely outside your experience

Use this structure:

Acknowledge → Relate → Explain → Learn

For example:

Interviewer:
"Have you implemented event-driven architecture using Kafka?"

You:

"I haven't implemented Kafka directly in a production project. My experience has primarily been around .NET, Azure and Cosmos DB. However, I understand the event-driven approach and the role of messaging systems in decoupling services. I've worked with similar concepts in cloud-based applications, so I believe I could learn Kafka quickly if the project requires it."

That's a much stronger response than:

"No, I don't know Kafka."

---

8. Most important: DON'T bluff

For senior positions, interviewers often ask **follow-up questions**.

Suppose you say:

"Yes, I've worked extensively with Azure Service Bus."

They may immediately ask:

"What's the difference between PeekLock and ReceiveAndDelete?"

If you don't know, you've created a credibility problem.

Instead, say:

"I have limited hands-on experience with Service Bus. My stronger Azure experience is with Cosmos DB, Functions, Storage, Managed Identity and Azure DevOps."

That actually **strengthens your credibility**.

Your resume specifically highlights Azure Cosmos DB, Azure Functions, Azure Storage, Managed Identity, Azure RBAC, Azure DevOps, Bicep and ARM Templates.

🏆 5 sentences you should memorize

These five responses will cover most situations:

When you forgot

"I'm familiar with it, but I can't recall the exact details at the moment."

When you don't know

"I haven't worked directly with that, so I don't want to give you an incorrect answer."

When you know part of it

"I haven't implemented it myself, but my understanding of the concept is..."

When you need clarification

"Could you please clarify the scenario you're referring to?"

When you need thinking time

"That's a good question. Let me think through the best approach."

⭐ One powerful technique for your interviews

When you don't know the exact answer, **show how you would find the answer**.

For example:

Interviewer:
"How would you troubleshoot a sudden performance problem in a Cosmos DB application?"

Even if you don't remember a particular Cosmos DB feature, you can say:

"I would first try to identify whether the issue is coming from the application layer, API layer, or Cosmos DB itself. I would check application logs and Azure monitoring metrics, look at request latency and throttling, review the query patterns and partition-key behavior, and then identify whether the problem is related to RU consumption, inefficient queries, or an application-side bottleneck. Based on the findings, I would implement the appropriate optimization and validate it through monitoring."

That demonstrates **engineering thinking**, even if you don't remember every technical detail.

Remember this:

Don't try to prove that you know everything.

For a **12+ year Senior .NET/Azure developer**, the interviewer is also evaluating:

How you think → How you troubleshoot → How you communicate → How you handle uncertainty → How you make technical decisions.

Those qualities are particularly important for the **Engineering Lead** type of role you're targeting.

if they say "any questions to ask?" then what should I ask?

Yes. This question is an **important part of the interview**, especially for a **Senior .NET / Azure / Engineering Lead** position.

When the interviewer asks:

"Do you have any questions for us?"

Don't say only:

❌ "No, I don't have any questions."

Instead, ask **2–3 thoughtful questions** that show you're interested in the **role, technology, team, and expectations**.

⭐ Best Questions for Your Profile

Since you're targeting **Senior .NET / Azure Developer / Engineering Lead** roles, I recommend these.

1. Ask about the role

"Could you please tell me what the key responsibilities and expectations are for this role during the first 3 to 6 months?"

This is probably the **best question** to ask.

It shows that you're already thinking about how you can contribute.

---

2. Ask about the technology stack

"Could you tell me more about the current technology stack and architecture of the application? Specifically, how extensively are .NET, Azure and Angular used?"

This is particularly relevant to your profile because your strengths are **C#, .NET, Web API, Angular and Azure**.

---

3. For an Engineering Lead position

If this is the Engineering Lead role you mentioned earlier, ask:

"What are the expectations from the Engineering Lead in terms of technical ownership, architecture decisions, mentoring, and team management?"

This is an **excellent question** because it helps you understand whether the position is primarily technical or people-management oriented.

---

4. Ask about the team

"Could you tell me a little about the team structure and how the development, QA, DevOps, and architecture teams work together?"

This demonstrates that you understand software development is a **team effort**, not just coding.

---

5. Ask about the project

Since your recent experience involves **Microsoft and Azure Cosmos DB**, you can ask:

"What are the major technical challenges the team is currently working on, particularly around scalability, Azure, or application performance?"

This can lead to a very good technical discussion.

---

6. Ask about success

This is one of my favorite questions for senior candidates:

"How would you define success for someone in this position after the first six months?"

It makes you sound **senior, responsible, and outcome-oriented**.

🎯 If You Have Time for Only 2 Questions

Ask these:

Question 1:

"What are the key expectations from this role during the first 3 to 6 months?"

Question 2:

"What are the major technical challenges the team is currently facing, and how could someone in this role contribute to solving them?"

These two questions are enough.

👩‍💼 If the Interviewer Is an HR Manager

Ask more about the **role and organization**:

"Could you tell me more about the team structure and the role's responsibilities?"

"What are the next steps in the interview process?"

"What does the organization consider success in this position?"

Avoid asking about salary/leave/benefits **in the first technical interview** unless HR specifically brings them up.

👨‍💻 If the Interviewer Is a Technical Manager

Ask technical/leadership questions:

"Could you explain the current application architecture and technology stack?"

"What are the biggest technical challenges the team is currently facing?"

"How much involvement would this role have in architecture and design decisions?"

"What are the expectations around code reviews, mentoring, and technical leadership?"

🏗️ If the Interviewer Is an Engineering Manager

For an **Engineering Lead** interview, this would be my preferred set:

1. "What are the key expectations from this Engineering Lead role during the first 3 to 6 months?"

2. "How is the team structured, and what would be the expected level of involvement in technical decisions, mentoring, and stakeholder management?"

3. "What are the biggest technical or delivery challenges the team is currently facing?"

You don't need to ask all three. **Pick two.**

⭐ A Very Strong Closing Question

At the end, you can ask:

"Based on our discussion today, is there any area of my experience that you would like me to explain in more detail?"

This is a **very powerful question**.

Why?

If the interviewer has a concern about something—perhaps **Angular, Azure, leadership, architecture, or team handling**—they have an opportunity to tell you.

You can then address it before the interview ends.

For example:

Interviewer:
"I'm not completely clear about your leadership experience."

You can respond:

"Sure. Let me explain the situations where I took technical ownership, worked with team members, coordinated with stakeholders, and helped resolve production issues..."

That gives you a chance to **remove doubts before leaving the interview**.

🚫 Questions I Would Avoid Initially

Don't make your first question:

❌ "What is the salary?"

❌ "How many leaves do I get?"

❌ "Is work from home available?"

❌ "How soon can I get promoted?"

❌ "How many hours do I have to work?"

These are legitimate questions, but they're generally better discussed during the **HR/offer stage**.

🏆 Your Interview Closing Script

For your upcoming interviews, you can simply say:

"Yes, I have a couple of questions. First, could you tell me what the key expectations are for this role during the first 3 to 6 months? And secondly, what are the major technical challenges the team is currently facing?"

After they answer:

"Thank you. That gives me a much better understanding of the role."

Then, if appropriate:

"Before we close, is there anything about my experience or background that you would like me to elaborate on?"

That's a **very strong way to finish a Senior .NET/Azure interview.**

while interview how should I behave, sitting and eye contact etc

Absolutely. For a **Senior .NET / Azure / Engineering Lead** interview, your body language should communicate:

Confidence + calmness + professionalism + listening + technical maturity.

You don't need to look overly serious. The goal is to appear **comfortable and confident**, not rehearsed.

1. 🪑 How to Sit

Sit upright with your back reasonably straight.

Good posture

- Keep your **back straight** and shoulders relaxed.
- Keep both feet comfortably on the floor.
- Keep your hands naturally on the table or your lap.
- Lean **slightly forward** when listening.
- Sit comfortably—not rigidly.

Think:

"I'm having a professional conversation, not attending an exam."

Avoid

❌ Slouching
❌ Leaning too far back
❌ Constantly changing your sitting position
❌ Crossing your arms for long periods
❌ Putting your feet on the chair or moving them constantly

---

2. 👀 Eye Contact — Very Important

You don't need to stare continuously.

A good pattern is:

Look at interviewer → speak → briefly look away while thinking → return eye contact.

When they're speaking:

👀 Look at them and listen.

When you're answering:

👀 Maintain comfortable eye contact for a few seconds at a time.

If there are multiple interviewers, distribute your attention.

For example:

Interviewer 1 asks the question → look primarily at Interviewer 1.

While answering:

Look at Interviewer 1 → occasionally look at the other interviewers → return to Interviewer 1.

Avoid

❌ Staring continuously
❌ Looking at the floor while answering
❌ Looking at the ceiling for the entire answer
❌ Looking at your phone
❌ Constantly looking at your own screen

---

3. 🙂 Facial Expression

Keep a **neutral and approachable expression**.

You don't need to smile continuously.

When they introduce themselves:

😊 Small smile + nod.

When they explain something:

Nod occasionally to show you're listening.

When they ask a difficult question:

Don't immediately look worried.

Instead:

Pause → think → respond calmly.

For example:

"That's a good question. Let me think about that for a moment."

This looks much better than:

😨 "Oh... I don't know..."

---

4. 🤝 How to Greet Them

For an online interview:

"Good morning, [Name]. It's nice to meet you. Thank you for taking the time to speak with me today."

If you don't know their name:

"Good morning. Thank you for giving me this opportunity. It's nice to meet you."

Don't start talking immediately after joining.

Wait for the interviewer to initiate if they're still getting settled.

---

5. 🗣️ How to Speak

This is particularly important for you because you're preparing for a **Senior-level position**.

Speak slightly slower than normal.

Don't rush because you're nervous.

Instead of:

"IworkedonCsharpdotnetwebapiandazureandcosmosdb..."

Say:

"In my recent project, I worked extensively with **C#, .NET Web API, Azure, and Cosmos DB**."

Pause between important points.

Use this pattern:

Point → Explanation → Example

For example:

"We used Azure Cosmos DB for storing application data. One of the key considerations was partitioning. We selected the partition key based on the application's access patterns to improve scalability and avoid unnecessary cross-partition queries."

This sounds much more senior than simply listing technologies.

---

6. ✋ What to Do With Your Hands

Don't worry too much about your hands.

Keep them:

- On your lap, or
- Naturally on the desk.

Small hand gestures while explaining something are perfectly fine.

For example, when explaining architecture:

"The request comes through the API layer, then we validate it, process the business logic, and finally interact with Cosmos DB."

Natural gestures can help.

Avoid

❌ Playing with a pen
❌ Touching your face repeatedly
❌ Adjusting your hair constantly
❌ Clicking your pen
❌ Tapping your fingers
❌ Constantly touching your keyboard

These can reveal nervousness.

---

7. 💻 For an Online Interview

Since many technical interviews are online, pay particular attention to these.

Camera

Position the camera approximately at **eye level**.

Don't put the laptop too low so the interviewer is looking up at you.

Background

Keep it:

- Clean
- Simple

- Professional

Lighting

Have light in front of you rather than behind you.

Camera

Look into the camera **occasionally**, especially when making an important point.

But don't force yourself to stare at the camera continuously.

When listening, it's natural to look at the interviewer's face on the screen.

---

8. 🧠 What to Do When You Don't Know an Answer

This is where your body language becomes very important.

Suppose they ask:

"Explain the difference between Azure Service Bus and Event Grid."

And you don't remember.

Don't immediately:

😨 Look down → nervous smile → "I don't know."

Instead:

Pause → maintain calm eye contact → respond professionally.

"I'm familiar with both services, but I don't want to give you an inaccurate comparison without verifying the exact details. My understanding is that Service Bus is primarily used for reliable enterprise messaging, while Event Grid is more event-notification oriented."

That's excellent interview behavior.

---

9. 📝 Can You Take Notes?

Yes. Absolutely.

For a senior interview, having a small notebook beside you is perfectly professional.

You can write down:

- Interviewer's name
- Important requirements
- Questions you want to ask
- Architecture components
- Something you need to come back to

For example:

"I'll come back to that point after I explain the API architecture."

This can be very useful during system-design discussions.

---

10. 🧩 During Technical Questions

Don't jump into coding immediately.

If they ask:

"How would you design this application?"

First clarify the requirements.

Say:

"Before I propose the design, I'd like to clarify a couple of requirements."

Then ask about:

- Expected users
- Request volume
- Availability
- Security
- Data size
- Performance
- Scalability

Then explain your design.

This is particularly important for an **Engineering Lead** interview.

---

11. 🎯 When They Challenge Your Answer

Suppose the interviewer says:

"I don't think that approach would work."

Don't become defensive.

Instead:

"That's a good point. Let me reconsider that."

Then:

"If scalability is the primary concern, I would modify the approach by..."

This demonstrates **technical maturity**.

Never argue just to prove that you're right.

---

12. 🚫 Things You Should NOT Do

Avoid	Do instead
Slouching	Sit upright
Looking at the floor	Comfortable eye contact
Speaking too fast	Speak slowly and clearly
Constant smiling	Natural expression
Saying "I don't know" abruptly	Explain what you do know
Interrupting	Let them finish
Arguing	Discuss alternatives
Long memorized answers	Conversational answers
Reading your resume	Explain your experience naturally
Excessive hand movement	Natural gestures
Checking phone	Keep phone away

⭐ Your Ideal Interview Behavior

Imagine you're sitting like this:

Posture:
🪑 Upright + relaxed

Face:
😊 Calm + approachable

Eyes:

👀 Comfortable eye contact

Hands:

✋ Relaxed and still unless gesturing

Voice:

🗣️ Clear + slightly slower than normal

Mindset:

🧠 "I'm discussing my experience with another professional."

Not:

"I must answer every question perfectly."

One Final Tip for You

Because you have **12+ years of experience**, don't behave like you're trying to impress the interviewer by proving that you know every technology.

Your strongest presentation is:

"I have solid hands-on experience. I understand why we made particular technical decisions. I've faced production problems and solved them. And when I don't know something, I know how to investigate and find the right solution."

That is the mindset of a **Senior Developer / Engineering Lead**.

Before the interview, remember these 5 things:

Sit straight → Look at them → Listen completely → Pause before answering → Speak clearly.

If you practice just these five behaviors, your confidence will noticeably improve.

I am not perfect in speaking general english, while I speak in english so many pauses happended, trying to phrase the sentense in my mind, getting nervous while speaking, hesitation, speaking haltingly. how should I overcome these

Absolutely. And first, I want you to know something important:

Your problem is not that you "don't know English." From the way you've communicated with me, you can clearly express technical ideas. Your bigger issue is **speaking fluently under pressure**—you are translating/forming the sentence in your mind while simultaneously trying to speak.

This is very common among experienced technical professionals, especially in interviews.

The good news is: **you can improve this significantly with practice.**

1. Don't try to speak perfect English

This is the biggest change I want you to make.

When you speak, you're probably doing this:

Think in your native language → translate to English → check grammar → construct sentence → speak

That creates:

Pause → hesitation → nervousness → more thinking → longer pause

Instead, aim for:

Simple thought → simple English → speak

For example, don't try to say:

"In the context of the application architecture, we were responsible for implementing a scalable mechanism which would facilitate..."

Say:

"In this project, we needed a scalable solution. So, we used Azure Cosmos DB."

Simple English sounds **more confident** than complicated English with many pauses.

---

2. Use "speaking chunks"

Don't construct every sentence word by word.

Memorize useful **sentence patterns**, not entire answers.

For your .NET interviews, learn patterns such as:

Starting an answer

"In my experience..."

"In my recent project..."

"The approach we followed was..."

"There are two important points here..."

"Let me explain this with an example."

Explaining something

"The main reason for this is..."

"The advantage of this approach is..."

"One challenge we faced was..."

"To solve this problem, we..."

Giving an example

"For example, in my recent project..."

"One example from my experience is..."

When you need time

"That's a good question. Let me think about it for a moment."

"Let me explain what I understand about this."

"I haven't worked on that directly, but my understanding is..."

These phrases give your brain **time to organize the next sentence**.

---

3. Don't be afraid of pauses

This is extremely important.

You currently think:

"I'm pausing. The interviewer will think my English is bad."

That thought creates nervousness.

Instead, understand:

A short pause is normal.

A professional speaker might say:

"There are... [pause] two main reasons for this."

That sounds thoughtful.

But:

"There are... ummm... actually... basically... I mean..."

sounds uncertain.

So replace **filler words** with **silence**.

Instead of:

"Actually... basically... you know... I think..."

Say:

"Let me think about that for a moment."

Then pause.

That's professional.

---

4. Speak slower

I strongly recommend that you **reduce your speaking speed by about 20–30%** during interviews.

You don't need to speak like a native English speaker.

Your objective is:

Clear > Fast

For example, don't say:

"IworkedonAzureCosmosDBandweusedmanagedidentitybecause..."

Instead:

"In my recent project, I worked extensively with **Azure Cosmos DB**.
We also used **Managed Identity** for secure access.
One of the benefits was that we didn't have to manage credentials in the application."

Notice the short sentences.

That is much easier to speak.

---

5. Use the PREP technique

This is one of the best techniques for you.

P — Point

Give your answer.

R — Reason

Explain why.

E — Example

Give an example.

P — Point

Conclude.

For example:

Question:
"What is dependency injection?"

You:

"Dependency Injection is a design pattern used to provide dependencies to a class rather than creating them inside the class.

The main benefit is that it reduces tight coupling and makes the code easier to test.

For example, in an ASP.NET Core application, instead of creating a repository directly inside a service, we inject the repository through the constructor.

So overall, dependency injection improves maintainability, testability, and flexibility."

You don't have to think about the entire answer at once.

Think:

Point → Reason → Example → Conclusion

---

6. Don't memorize complete interview answers

This is particularly important for your interview preparation.

If you memorize:

"Dependency injection is a software design pattern in which an object receives other objects that it depends on..."

and then forget one sentence, your brain may suddenly stop.

Instead, remember **keywords**:

DI → dependency → loose coupling → testability → constructor injection → example

Then speak naturally around those keywords.

This will dramatically reduce your hesitation.

---

7. Practice speaking, NOT reading

Reading interview answers silently won't solve your problem.

You need to **speak out loud**.

Every day, take one technical question.

For example:

"Explain your recent project."

Look at only 4–5 keywords:

Microsoft → Cosmos DB → .NET → Azure → production support

Then speak for 2 minutes.

Don't read a prepared paragraph.

This trains your brain to **generate English naturally**.

---

8. Your daily 20-minute English practice

I recommend this routine for you.

🕐 **5 minutes — Read aloud**

Take a technical article or your resume.

Read it **slowly and clearly**.

Don't worry about accent.

Focus on pronunciation and rhythm.

🕐 **5 minutes — Talk about yourself**

Without looking at your resume, answer:

"Tell me about yourself."

Record yourself.

Don't restart if you make a mistake.

Keep speaking.

🕐 **5 minutes — Technical explanation**

Pick one topic:

- C#
- .NET
- Web API
- Angular
- Azure
- Cosmos DB
- Design Patterns
- Microservices

Explain it as if you're talking to an interviewer.

🕐 **5 minutes — General English**

Talk about something simple:

"What did I do today?"

"What did I work on in my previous project?"

"What are my strengths?"

"What kind of job am I looking for?"

This develops **general conversational fluency**, which is different from technical knowledge.

---

9. One exercise I especially recommend for you

The "No Restart" Exercise

Set a timer for **2 minutes**.

Choose a topic:

"My recent project"

Start speaking.

If you make a grammar mistake:

DO NOT STOP.

If you forget a word:

Use another word.

If you make a bad sentence:

Continue.

For example:

"In my recent project, I worked on... actually, the project was related to... Microsoft... Azure Cosmos DB. My responsibility was mainly... I worked on the API... and also... Azure services..."

Don't stop.

Your goal is **continuous communication**, not perfect grammar.

After 2 minutes, stop.

Then identify **only 2 mistakes**.

Don't try to correct 20 things at once.

---

10. Learn "rescue sentences"

These are extremely useful during interviews.

When you get stuck:

Forgot a word

"Let me put that another way."

Need time

"Let me think about that for a moment."

Lost your sentence

"Let me start that again."

Want to clarify

"What I mean is..."

Don't remember exactly

"I don't recall the exact details, but my understanding is..."

Don't know

"I haven't worked with that directly, so I don't want to give you an incorrect answer."

These phrases prevent a small hesitation from becoming a **complete breakdown in your speaking**.

---

11. Your nervousness is probably coming from one expectation

You may be thinking:

"I have to speak perfect English."

Change that to:

"I only need to communicate my experience clearly."

You are a **12+ year experienced developer**.

The interviewer isn't hiring you to be an English teacher.

They want to understand:

- What have you worked on?
- How well do you understand .NET?
- How strong are you in Azure?
- Can you solve problems?
- Can you communicate with the team?
- Can you explain technical decisions?
- Can you handle responsibility?

Your English only needs to be **clear enough to communicate these things effectively**.

---

12. Don't compare yourself with native speakers

You may hear someone speaking:

"Basically, what we're trying to achieve here is to leverage..."

very quickly and naturally.

Don't try to copy that.

Your goal should be:

Clear + structured + confident

For example:

"There were two challenges in this project.

The first was performance.

The second was scalability.

We addressed these by optimizing the API and improving our Cosmos DB partitioning strategy."

That sounds **professional**, even though the English is simple.

---

13. Your interview answer should have a rhythm

Try this:

Short sentence.
Pause.
Short sentence.
Pause.
Example.

For example:

"I have around 12 years of experience in .NET development.
My primary expertise is C#, .NET, Web API and Angular.
I also have strong experience with Azure.
In my recent project, I worked on Microsoft Azure Cosmos DB.
My responsibilities included development, troubleshooting and production support."

Don't try to say everything in one long sentence.

🎯 **I want to help you practice this differently**

Since your immediate goal is **interview preparation**, we can make this a practical speaking exercise rather than just giving you advice.

I'll act as your **interviewer**.

I'll ask you **one question at a time**.

You answer in your own English—even if it has grammar mistakes, pauses, or incomplete sentences.

Then I'll give you:

1. **Grammar corrections**
2. **More natural English**
3. **Better professional wording**
4. **Feedback on hesitation/structure**
5. **A simple version you can actually speak**
6. **A better senior-level version**
7. **The next interview question**

And importantly, **I won't expect perfect English from you**. The goal is to make *your existing English* more fluent and confident.

Let's start with the most important question:

Interviewer:

"Good morning, Satheesh. Could you please introduce yourself and tell me about your professional experience?"

🎤 **Now answer this in your own words. Don't prepare or copy an answer. Just speak/type the way you would actually say it in an interview.**

while interview I am forgetting the concepts that I have read before, I am unable to recall well known concepts that I read many times. what is the solution for this

Yes. What you're experiencing is **very common during interviews**, especially when you have studied many topics. The problem usually isn't that you've forgotten the knowledge permanently. It's that **you can recognize the concept when reading it, but you can't retrieve it quickly when you're under pressure**.

For example, you may have read dependency injection 10 times. When you see:

"What is Dependency Injection?"

you think:

"I know this... I studied this... why can't I remember the answer?"

This is a **recall problem**, not necessarily a knowledge problem.

1. Stop rereading; start active recall

This is the biggest change I recommend.

❌ **Current approach**

You read:

Dependency Injection → definition → benefits → example → types

You feel:

"Yes, I understand this."

Then you move to the next topic.

But during the interview, there is nothing in front of you to trigger the memory.

✅ **New approach**

Close your notes and ask yourself:

"What is Dependency Injection?"

Try to answer **without looking**.

Even if you remember only:

"DI... dependency... loose coupling..."

That's good.

Then open your notes and check what you missed.

This process is called **active recall**, and it's much more useful for interview preparation than repeatedly reading the same material.

---

2. Use the "5-point recall" method

For every interview concept, don't memorize a long answer.

Remember just **5 things**:

1. What?

What is it?

2. Why?

Why do we use it?

3. How?

How does it work?

4. Example

Can I give a simple example?

5. When?

When would I use it?

For example:

Dependency Injection

What?

A technique for providing dependencies to a class instead of creating them inside the class.

Why?

Loose coupling and better testability.

How?

Constructor injection, primarily.

Example?

Inject IRepository into a service.

When?

When a class depends on another service/component.

That's enough to reconstruct a good interview answer.

You don't need to memorize a 2-minute paragraph.

---

3. Build "memory hooks"

Your brain remembers **connections** better than isolated information.

For example, remember:

DI → Loose Coupling → Testability → Constructor Injection

Then, when the interviewer asks:

"What is dependency injection?"

Your brain has a path to follow.

Similarly:

SOLID

S → Single Responsibility
O → Open/Closed
L → Liskov Substitution
I → Interface Segregation
D → Dependency Inversion

async/await

async → asynchronous method
await → wait without blocking the calling thread

Web API

HTTP → Controller → Service → Repository → Database

Cosmos DB

Container → Partition Key → RU → Query → Consistency

These keywords act as **retrieval triggers**.

---

4. Practice questions randomly

This is extremely important for you.

If you study:

C# → .NET → Web API → Azure → Angular

in that order every day, your brain starts remembering the **sequence**, not necessarily the concepts.

In an interview, questions come randomly.

You might get:

Cosmos DB

then:

Dependency Injection

then:

Angular

then:

Design Patterns

then:

async/await

So practice randomly.

For example, write 50 questions on paper/cards and randomly pick one.

Ask yourself:

"Explain the difference between IEnumerable and IQueryable."

Answer.

Then:

"What is a partition key in Cosmos DB?"

Answer.

Then:

"What is middleware in ASP.NET Core?"

Answer.

This trains your brain for the **actual interview environment**.

---

5. Use spaced repetition

Don't study a concept once and leave it.

Use this schedule:

Day 1

Learn the concept.

Day 2

Recall it without notes.

Day 4

Recall it again.

Day 7

Recall it again.

Day 14

Recall it again.

Day 30

Recall it again.

You don't need to spend an hour each time.

Even **2–5 minutes of recall** can be valuable.

---

6. Speak the answers aloud

This is especially important **for you**, because you have already mentioned that you experience pauses and hesitation while speaking English.

You actually have **two problems interacting with each other**:

Technical recall problem

-

English speaking/fluency pressure

=

"I know this concept, but I can't remember it or explain it."

Therefore, don't just write answers.

Speak them.

For example:

"Dependency injection is a design technique where dependencies are provided to a class instead of creating them inside the class..."

Record yourself.

Listen back.

Then try again.

The second attempt will usually be easier.

---

7. Use the "3-level answer" technique

Don't force yourself to produce a perfect answer immediately.

Level 1 — 15 seconds

Give the definition.

"Dependency Injection is a technique where dependencies are provided to a class rather than created inside it."

Level 2 — 30 seconds

Add the purpose.

"It helps us achieve loose coupling and makes the code easier to test and maintain."

Level 3 — 60–90 seconds

Add an example.

"For example, in ASP.NET Core, we can register an interface and its implementation in the DI container and inject it into a service through the constructor..."

This is **much easier for your brain** than trying to remember a complete 90-second answer immediately.

---

8. Don't try to remember every detail

This is probably one of the biggest mistakes you're making.

You may be trying to remember:

Definition
Advantages
Disadvantages
Syntax
Internal implementation
Edge cases
Version differences
Interview examples
Real-world examples

for every topic.

That's too much.

For a first response, remember:

Definition + Why + Example

Then let the interviewer ask follow-up questions.

For example:

Interviewer:
"What is middleware?"

You:

"Middleware is a component in the ASP.NET Core request pipeline that can inspect or modify HTTP requests and responses. We commonly use middleware for things like exception handling, authentication, logging, and request processing."

That's enough.

If they ask:

"How does middleware work?"

Then go deeper.

---

9. Use your real project to remember concepts

This will be **especially effective for you**.

Instead of memorizing:

"Azure Cosmos DB is a globally distributed NoSQL database..."

connect it to your experience:

Microsoft project → Cosmos DB → containers → partitioning → queries → RU → production troubleshooting

Now the concept has a real context.

Similarly:

Web API

Think:

My project → API → Controller → Service → Repository → Cosmos DB

Dependency Injection

Think:

ASP.NET Core → interface → service → constructor injection

Azure Functions

Think:

Backend processing → trigger → function → Cosmos/Storage

Managed Identity

Think:

Azure application → identity → RBAC → secure access → no credentials

You will remember **experiences much more easily than textbook definitions**.

---

10. What to do when your mind goes blank in the interview

This is very important.

Suppose the interviewer asks:

"What are the SOLID principles?"

And suddenly your mind is blank.

Don't panic.

Take a breath.

Say:

"Yes, I'm familiar with SOLID principles. Let me recall them one by one."

Then start with whatever comes to mind:

"S is Single Responsibility..."

Once you say the first one, your memory often starts retrieving the others.

If you remember only three:

"I can recall Single Responsibility, Open/Closed, and Dependency Inversion. I'm missing the exact order of the remaining principles at the moment."

That's much better than freezing.

---

11. Use a "brain dump" before the interview

About **5–10 minutes before the interview**, don't try to learn anything new.

Take a sheet of paper and write:

C#

OOP | async/await | LINQ | delegates | generics | exceptions

.NET

DI | Middleware | Configuration | Logging | Authentication

Web API

REST | HTTP | JWT | Status Codes | Versioning | Exception Handling

Azure

Cosmos DB | Functions | Storage | RBAC | Managed Identity | DevOps

Architecture

SOLID | Design Patterns | Microservices | Caching | scalability

This activates your memory.

Then stop studying.

---

12. A better preparation strategy for you

Given your experience, **don't try to cover 500 interview questions**.

I'd rather have you master **100 important questions** than superficially read 500.

For each question, create a small card:

QUESTION:

What is Dependency Injection?

WHAT?

Provide dependencies from outside.

WHY?

Loose coupling + testability.

HOW?

Constructor injection.

EXAMPLE?

IService → Service implementation.

REAL PROJECT?

ASP.NET Core application.

That's all.

Then close the card and explain it aloud.

🧠 **Your biggest mindset change**

When you forget something during an interview, **don't immediately conclude:**

"I don't know this."

Instead ask yourself:

"What do I remember about this?"

There is a huge difference.

Your memory may give you:

"It's related to loose coupling..."

Then:

"It's used in ASP.NET Core..."

Then:

"Constructor injection..."

And suddenly the whole concept comes back.

Give your brain a starting point.

🎯 I recommend we do this as part of your interview preparation

Rather than you continuing to read hundreds of answers, I can act as your **mock interviewer + recall coach**.

I'll ask you questions **randomly** from:

C# → .NET → Web API → Angular → Azure → Cosmos DB → SQL → Design Patterns → Microservices → System Design

You answer **without looking at your notes**.

If you forget, I'll give you a small hint rather than immediately giving you the answer.

Then I'll show you:

- ✅ What you remembered correctly
- ❌ What you missed
- 🧠 A simple memory hook
- 🎤 A natural way to answer it verbally
- ⭐ A senior-level answer
- 📝 What to remember for the next interview

This will simultaneously improve **technical recall, English fluency, confidence, and interview performance**—which are the areas you've been working on.